

1/ BFS Lab Assignment 04

=>

```
#include <iostream>
using namespace std;

const int MAX = 100;

// Queue using array
class Queue {
    int arr[MAX];
    int front, rear;

public:
    Queue() {
        front = rear = -1;
    }

    bool isEmpty() {
        return front == -1;
    }

    bool isFull() {
        return (rear + 1) % MAX == front;
    }

    // Adding new item
    void enqueue(int val) {
        if (isFull()) {
            cout << "Queue is full\n";
            return;
        }
        if (isEmpty()) {
            front = rear = 0;
        } else {
            rear = (rear + 1) % MAX;
        }
        arr[rear] = val;
    }

    // Removing item from the front
```

```

int dequeue() {
    if (isEmpty()) {
        cout << "Queue is empty\n";
        return -1;
    }
    int val = arr[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX;
    }
    return val;
}

// Display the queue
void display() {
    if (isEmpty()) {
        cout << "Queue is empty\n";
        return;
    }
    cout << "Queue: ";
    int i = front;
    while (true) {
        cout << arr[i] << " ";
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    cout << endl;
}

};

// Input edges and build adjacency matrix
void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    // Initialize matrix to 0
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;

    cout << "Enter edges (source destination):\n";
    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        if (u >= 0 && u < n && v >= 0 && v < n) {

```

```

        adj[u][v] = 1;
    } else {
        cout << "Invalid nodes. Try again.\n";
        i--;
    }
}
}

```

// Print adjacency matrix

```

void printAdjMatrix(int adj[MAX][MAX], int n) {
    cout << "\nAdjacency Matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << adj[i][j] << " ";
        }
        cout << endl;
    }
}

```

```

int main() {
    int n, e, s;    // n=number of nodes, e=number of edges, s=start node
    int adj[MAX][MAX];

```

/// Input number of nodes and edges

```

cout << "Enter number of nodes: ";
cin >> n;
cout << "Enter number of edges: ";
cin >> e;

```

// To track visited nodes and Initialize visited array to 0

```

int visited[n];
for (int i = 0; i < n; i++) {
    visited[i] = 0;
}

```

// Input edges to adjacency matrix

```

inputEdgesToAdjMatrix(adj, n, e);
printAdjMatrix(adj, n);    //Print adjacency matrix

```

//BFS traversal using Queue

```

Queue q, out;

```

```

cout<<endl;

```

// BFS starting input

```

cout<<"Enter start node: ";
cin>>s;

/// Check if start node is valid
while (s>=n || s<0) {
    cout << "Invalid start node. Enter again: ";
    cin >> s;
}

q.enqueue(s);    // Enqueue start node

// BFS traversal
while (!q.isEmpty()) {    //Check if queue is empty
    s = q.dequeue();    // Dequeue a node from queue
    visited[s] = 1;    // Mark the node as visited
    out.enqueue(s);    // Enqueue the node to output queue

    // Check all adjacent nodes
    for (int i = 0; i < n; i++){
        // Check if the node is adjacent and not visited
        if (adj[s][i] == 1 && visited[i] == 0) {
            q.enqueue(i);    // Enqueue the not visited node
            visited[i] = 1;    // Mark the node as visited
        }
    }

    // Check if queue is empty and is there any unvisited nodes
    if(q.isEmpty()){
        for(int j;j<n;j++){
            // Check if there are any unvisited nodes
            if(visited[j] == 0){
                q.enqueue(j);    // Enqueue the unvisited node
                break;
            }
        }
    }
}

// Print the output queue
cout<< "output: ";out.display();

return 0;
}

```

2/ Lab Assignment 05

=>

```
#include <iostream>
using namespace std;
```

```
const int MAX = 100;
const int INF = 100000000;
```

```
class FixTraker{
public:
    int f;
    int d;
    FixTraker(){
        f=0;
        d=INF;
    }
    set(int v){
        f=1;
        d=v;
    }
};
```

```
// Input edges and build adjacency matrix
void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    // Initialize matrix to 0
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++){
            if(i==j) adj[i][j] = 0;
            else adj[i][j] = INF;
        }
}
```

```
cout << "Enter edges (source destination wight):\n";
for (int i = 0; i < e; i++) {
    int u, v, w;
    cin >> u >> v >> w;
    if ( u !=v && u >= 0 && u < n && v >= 0 && v < n) {
        adj[u][v] = w;
    } else {
        cout << "Invalid nodes. Try again.\n";
    }
}
```

```

        i--;
    }
}
}

// Print adjacency matrix
void printAdjMatrix(int adj[MAX][MAX], int n) {
    cout << "\nAdjacency Matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << adj[i][j] << " ";
        }
        cout << endl;
    }
}
}

```

```

int main() {
    int n, e, s, f=1;
    int adj[MAX][MAX];

    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges less then "<<(n*(n-1))+1<<" : ";
    cin >> e;

    while(e>n*(n-1)){
        cout << "error! Enter number of edges less then "<<(n*(n-1))+1<<" : ";
        cin >> e;
    }

    inputEdgesToAdjMatrix(adj, n, e);
    //printAdjMatrix(adj, n);

    cout << "Enter number of edges less then "<<n<<" : ";
    cin >> s;

    while(s>n){
        cout << "error! Enter number of edges less then "<<n<<" : ";
        cin >> s;
    }

    FixTraker t[n];
    t[s].set(0);
}

```

```

for(int i=1;i<n;i++){
    for(int j=0;j<n;j++){
        if(t[j].f==0 && t[j].d >(adj[s][j]+t[s].d)) t[j].d=adj[s][j]+t[s].d;
    }

    int minln=-1,min=INF;
    for(int j=0;j<n;j++){
        if(t[j].f==0 && min>t[j].d){
            min=t[j].d;
            minln=j;
        }
    }
    s=minln;
    t[s].f=1;
}

cout<<"result: \n[ ";
for(int i=0;i<n-1;i++){
    cout<<t[i].d<<" ";
}
cout<<t[n-1].d<<" ]";

return 0;
}

```